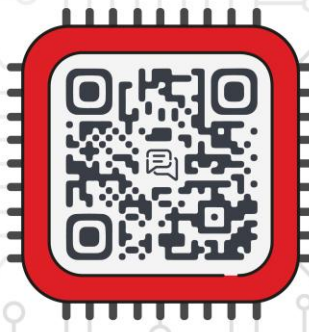




2



15



1800 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

تمارين على الأشجار الثنائية

د. عملي مشترك



RB Informatics; 16/04/2025

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

في المحاضرة السابقة تحدثنا عن :

- GRAPH THEORY CONCEPTS
- THE RELATIONSHIP BETWEEN GRAPHS AND TREES
- ROOTED TREES BASIC CONCEPTS
- BINARY TREES IN GENERAL AND HOW TO TRAVERSE IN IT

كان هنالك مجموعة من التمارين سنحلها قبل البدء.

التمرين الأول : البحث عن أكبر عنصر في شجرة ثنائية :

- فكرة الحل :** بالنسبة لكل عقدة ، نبحث عن أكبر قيمة بالشجرة اليسرى بالنسبة لها ، و نبحث عن أكبر قيمة بالشجرة اليمنى بالنسبة لها و ذلك يتم بشكل عودي .
ثم نقارن الإجابة القادمة من الابن اليسر و الابن الأيمن بقيمة العقدة الحالية و نأخذ الأكبر بينهم.

ملاحظة : عند الوصول لشرط التوقف (الوصول لعقدة null) سنقوم بإرجاع قيمة صغيرة جداً لن تؤثر على الإجابة .

```
1. class BinaryTree{
2.     Node root ;
3.     class Node {
4.         int value;
5.         Node left, right;
6.         Node(int value) {
7.             this.value = value;
8.             left = right = null;
9.         }
10.    }
```

```
11. public static int findMax(Node root) {
12.     if (root == null)
13.         //تابع يرجع أصغر قيمة ممكنة للعدد الصحيح برمجياً
14.         return Integer.MIN_VALUE;
15.     int maxLeft = findMax(root.left);
16.     int maxRight = findMax(root.right);
17.     return Math.max(root.value,
18.         Math.max(maxLeft, maxRight));
19. }
```

```

19. public class Main {
20.     public static void main(String[] args) {
21.         BinaryTree bt = new BinaryTree();
22.         bt.root = bt.new Node(10);
23.         bt.root.left = bt.new Node(20);
24.         bt.root.left.left = bt.new Node(50);
25.         bt.root.left.right = bt.new Node(40);
26.         bt.root.right = bt.new Node(39);
27.         bt.root.right.right = bt.new Node(2);
28.         int max = bt.findMax(bt.root);
29.         System.out.println("the max value is : " + max);
30.     }
31. }

```



التمرين الثاني: عكس العقد في شجرة.

- **فكرة الحل:** عند كل عقدة ننفذ عكس بين أبنائها و نستدعي التابع بشكل عودي على الأبناء حتى نصل لنهاية لعقدة null و هو شرط التوقف.

```

1. class BinaryTree{
2.     Node root ;
3.     class Node {
4.         int value;
5.         Node left, right;
6.         Node(int value) {
7.             this.value = value;
8.             left = right = null;
9.         }
10.    }

```

```

11. public void invertTree(Node root) {
12.     if (root == null)
13.         return;
14.     // مؤقت متغير تتطلب التبديل عملية
15.     Node temp = root.left;
16.     root.left = root.right;
17.     root.right = temp;
18.     invertTree(root.left);
19.     invertTree(root.right);

```

```

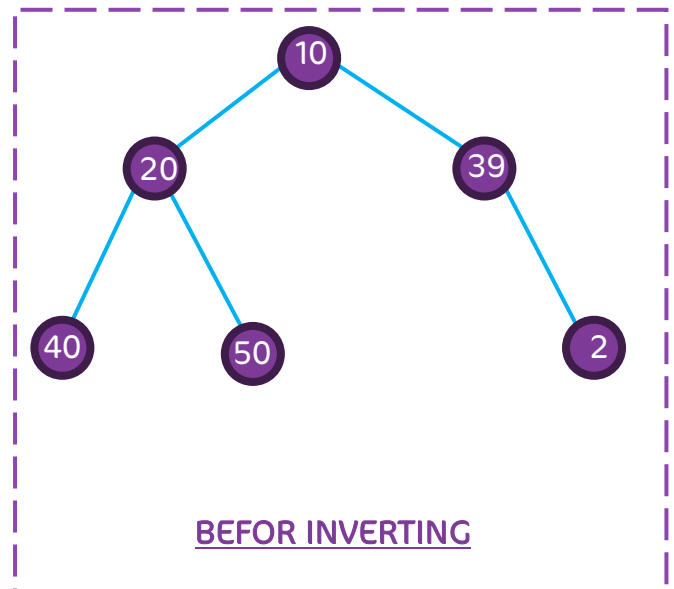
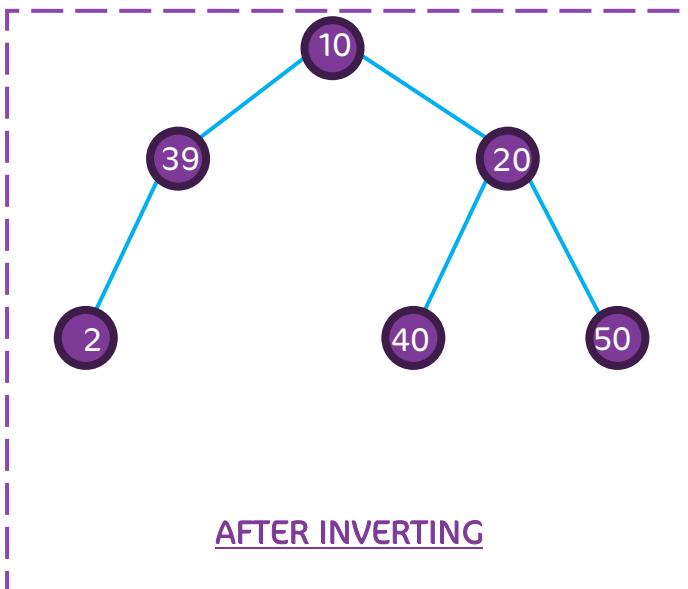
20. public void inorderTraversal(Node root) {
21.     if (root!=null) {
22.         inorderTraversal(root.left);
23.         System.out.print(root.value + " ");
24.         inorderTraversal(root.right);
25.     }
26. }
27.}

```

```
28. public class Main {
29.     public static void main(String[] args) {
30.         BinaryTree bt = new BinaryTree();
31.         bt.root = bt.new Node(10);
32.         bt.root.left = bt.new Node(20);
33.         bt.root.left.left = bt.new Node(50);
34.         bt.root.left.right = bt.new Node(40);
35.         bt.root.right = bt.new Node(39);
36.         bt.root.right.right = bt.new Node(2);
37.         System.out.println("in-order TRAVERSAL before inverting :");
38.         bt.inorderTraversal(bt.root);
39.         bt.invertTree(bt.root);
40.         System.out.println();
41.         System.out.println();
42.         System.out.println("in-order TRAVERSAL after inverting :");
43.         bt.inorderTraversal(bt.root);
44.     }
45. }
```

↪ بالنسبة للشجرة الحالية الخرج سيكون :

- in-order TRAVERSAL before inverting :
50 20 40 10 39 2
- in-order TRAVERSAL after inverting :
2 39 10 40 20 50



التمرين الثالث : تمثيل معادلة رياضية بشجرة و طباعة الناتج

■ **فكرة الحل :** لتمثيل المعادلة الرياضية بشكل شجرة

- كل عقدة لها ابنان لا بد أن تكون عبارة عن " عملية " و الابنان لا بد أن يكونا " أعداداً "
- لا يمكن لعقدة أن تملك ابناً وحيداً .
- العقد التي ليس لها أبناء لا بد أن تكون قيماً عددية (و نحقق هذا المطلب لكل شجرة جزئية من الشجرة الكلية بحيث سنقوم بعد قليل باستدعاء عودي).

بناء على ذلك فعند الوصول للأوراق سنجد قيماً عددية ، بعد ذلك سنقوم بالاستدعاء العودي التالي :
بالنسبة لكل عقدة نبحث عن قيمة العقد التي على يسارها و كذلك قيمة العقد التي على يمينها و ننفذ عليها العملية الموجودة في هذه العقدة ، شرط التوقف هو الوصول لورقة " لا تحمل عملية " عندئذ نعيد قيمة الورقة مباشرة .

```

1. class BinaryTree {
2.     Node root;
3.     class Node {
4.         String value;
5.         Node left, right;
6.         Node(String value) {
7.             this.value = value;
8.             this.left = this.right = null;
9.         }
10.    }

```

```

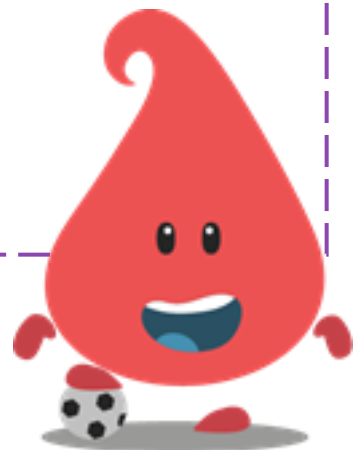
31. public class Main {
32.     public static void main(String[] args)
33.     {
34.         BinaryTree bt = new BinaryTree();
35.         //((2+4)*3)
36.         bt.root = bt.new Node("*");
37.         bt.root.left = bt.new Node("+");
38.         bt.root.right = bt.new Node("3");
39.         bt.root.left.left = bt.new Node("2");
40.         bt.root.left.right = bt.new Node("4");
41.         int result = bt.equation(bt.root);
42.         System.out.println("result: "+ result);
43.         // 18

```

```

11. public int equation(Node root) {
12.     if (root.left==null&& root.right==null)
13.         return Integer.parseInt(root.value);
14.     int leftVal = equation(root.left);
15.     int rightVal = equation(root.right);
16.     switch (root.value) {
17.         case "+":
18.             return leftVal+rightVal;
19.         case "-":
20.             return leftVal-rightVal;
21.         case "*":
22.             return leftVal*rightVal;
23.         case "/":
24.             return leftVal/rightVal;
25.         default:
26.             System.out.println("invalid
operator: " + root.value);
27.             return 0;
28.     }
29. }
30. }

```



الآن سنحل بعض التمارين على الشجرة الثنائية بحيث يصبح لدينا خلفية عن كيفية التعامل مع الأشجار الثنائية بصورة عامة :

التمرين الأول: حساب حجم الشجرة (number of nodes) tree size

- **فكرة الحل :** عند كل عقدة نجمع 1 (العقدة نفسها) + كل العقد إلى يمينها و كل العقد إلى يسارها بشكل عودي ، شرط التوقف هو الوصول لعقدة null عند ذلك نرجع قيمة 0 (لكي لا تؤثر على الناتج).

```
1. class BinaryTree {
2.     Node root;
3.     class Node {
4.         int value;
5.         Node left, right;
6.         Node(int value) {
7.             this.value = value;
8.             this.left = this.right = null;
9.         } }
10.    public int size(Node root) {
11.        if (root == null)
12.            return 0;
13.        return 1 + size(root.left) +
14.            size(root.right);
15.    } }
```

```
15. public class Main {
16.     public static void main(String[] args) {
17.         BinaryTree bt = new BinaryTree();
18.         bt.root = bt.new Node(10);
19.         bt.root.left = bt.new Node(20);
20.         bt.root.left.left = bt.new Node(50);
21.         bt.root.left.right = bt.new
22.             Node(40);
23.         bt.root.right = bt.new Node(39);
24.         bt.root.right.right = bt.new
25.             Node(2);
26.     } }
```

التمرين الثاني: حساب عدد الأوراق في الشجرة

tree leaves count(the number of nodes that have no children)

- **فكرة الحل :** عند كل عقدة نجمع جميع العقد الورقية يمينها و جميع العقد الورقية يسارها بشكل عودي ، شرط التوقف : يوجد شرطان عند الوصول لعقدة null فنرجع 0 (كون العقدة غير موجودة) عند الوصول لعقدة ليس لها أبناء نرجع 1 (هذه ورقة).

```
1. class BinaryTree {
2.     Node root;
3.     class Node {
4.         int value;
5.         Node left, right;
6.         Node(int value) {
7.             this.value = value;
8.             this.left = this.right = null; } }
```

```
9. public int countLeaves(Node root) {
10.     if (root==null) return 0;
11.     if (root.left==null && root.right== null)
12.         return 1;
13.     return countLeaves(root.left) +
14.         countLeaves(root.right);
15. }
```

```

15. public class Main {
16.     public static void main(String[] args) {
17.         BinaryTree bt = new BinaryTree();
18.         bt.root = bt.new Node(10);
19.         bt.root.left = bt.new Node(20);
20.         bt.root.left.left = bt.new Node(50);
21.         bt.root.left.right = bt.new Node(40);
22.         bt.root.right = bt.new Node(39);
23.         bt.root.right.right = bt.new Node(2);
24.         System.out.println(bt.countLeaves(bt.root)); //3
25.     }
26. }

```



التمرين الثالث: حساب عدد العقد الداخلية

internal nodes count (the number of nodes that have a child at least)

- **فكرة الحل :** عند كل عقدة في حال كان للعقدة ابن واحد على الأقل نقوم بجمع 1 (العقدة الحالية) و جميع العقد الداخلية على يسارها و جميع العقد الداخلية على يمينها و هذا يتم بشكل عودي شرط التوقف : الوصول لعقدة null .

```

1. class BinaryTree {
2.     Node root;
3.     class Node {
4.         int value;
5.         Node left, right;
6.         Node(int value) {
7.             this.value = value;
8.             this.left = this.right = null; }
9.     public int countInternalNodes ( Node
10.         root)
11.     if (root == null) return 0;
12.     if (root.left != null || root.right != null)
13.         return 1+
14.         countInternalNodes(root.left) +
15.         countInternalNodes(root.right);
16.     return 0; } }

```

```

14. public class Main {
15.     public static void main(String[] args) {
16.         BinaryTree bt = new BinaryTree();
17.         bt.root = bt.new Node(10);
18.         bt.root.left = bt.new Node(20);
19.         bt.root.left.left = bt.new Node(50);
20.         bt.root.left.right = bt.new Node(40);
21.         bt.root.right = bt.new Node(39);
22.         bt.root.right.right = bt.new Node(2);
23.         System.out.println(bt.countInternalNodes
24.             (bt.root) ); //3
25.     }

```


التمرين الرابع : حساب عدد العقد بكل مستوى من الشجرة tree nodes count at each level

- **فكرة الحل :** كما تعلمنا بالسابق هناك ما يسمى بـ (BFS) level order traversal و قد استخدمنا queue حتى نحققه بحيث كان يجب علينا تخزين أبناء كل عقدة ضمن الـ queue و ثم المرور على عناصر هذا الـ queue و بالتالي يمكننا معرفة عدد عقد مستوى ما بناء على ذلك انطلاقاً من المستوى الأول نضيف جميع أبناء الجذر الآن نعرف عدد عناصر المستوى الثاني و هو حجم الـ queue الآن نمر على عناصر المستوى الثاني و نضيف أبناءهم و نخرجهم من الـ queue بالتالي عند الانتهاء سنحصل على أبناء المستوى الثالث، الخ.....

```
1. import java.util.LinkedList;
2. import java.util.Queue;
3. class BinaryTree {
4.     Node root;
5.     class Node {
6.         int value;
7.         Node left, right;
8.         Node(int value) {
9.             this.value = value;
10.            this.left=this.right=null;
11.        }
12.    }
```

```
30. public class Main {
31.     public static void main(String[] args){
32.         BinaryTree bt = new BinaryTree();
33.         bt.root = bt.new Node(10);
34.         bt.root.left = bt.new Node(20);
35.         bt.root.left.left = bt.new Node(50);
36.         bt.root.left.right = bt.new Node(40);
37.         bt.root.right = bt.new Node(39);
38.         bt.root.right.right = bt.new Node(2);
39.         bt.root.right.right.right=bt.new Node(5);
40.         bt.countNodesAtEachLevel(bt.root);
41.         //Level 1: 1 //Level 2: 2
42.         //Level 3: 3 //Level 4: 1
43.     }
```

```
11. public void countNodesAtEachLevel
(Node root) {
12.     if (root == null) return;
13.     int level =1 ;
14.     Queue<Node>queue=new LinkedList<>();
15.     queue.add(root);
16.     while (!queue.isEmpty()) {
17.         int cnt = queue.size();
18.         System.out.print("Level
"+level+++" : "+cnt);
19.         for (int i = 0; i < cnt; i++) {
20.             Node node = queue.poll();
21.             if (node.left != null)
22.                 queue.add(node.left);
23.             if (node.right != null)
24.                 queue.add(node.right);
25.         }
26.         System.out.println();
27.     }
```



التمرين الخامس : احسب عدد العقد في مستوى معين tree nodes count at certain level

- **فكرة الحل :** نحتاج لمتغير يمثل المستوى نمرره كعامل للدالة الذي نرغب بالذهاب له و عند الوقوف عند كل عقدة نضيف الابن الأيمن و الابن الأيسر مع إنقاص قيمة متغير المستوى طبعاً هذا يتم كتابع عودي (أي: عندما نضيف أبناء عقدة نستدعي التابع على أن نجعل متغير المستوى أخفض من العقدة السابقة حتى نصل للمستوى المطلوب) ، شرط التوقف : عند الوصول لعقدة null نرجع صفراً أو عند الوصول للمستوى المطلوب نرجع (1) .

```

1.import java.util.LinkedList;
2.import java.util.Queue;
3.class BinaryTree {
4.    Node root;
5.    class Node {
6.        int value;
7.        Node left, right;
8.        Node(int value) {
9.            this.value = value;
10.           this.left = this.right = null;
11.        }
12.    }

```

```

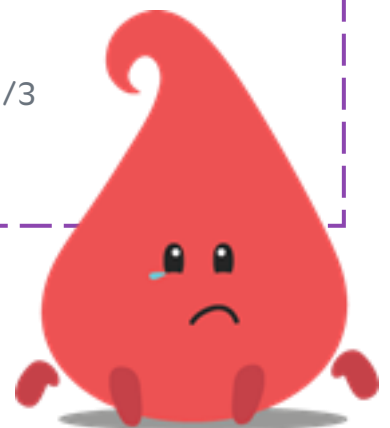
13.public int countNodesAtCertainLevel(Node root,
14.    int level) {
15.    if (root == null) return 0;
16.    if (level == 1) return 1;
17.    return countNodesAtCertainLevel(root.left,
18.    level - 1) + countNodesAtCertainLevel(root.right,
19.    level - 1);
20.}

```

```

1. public class Main {
2.     public static void main(String[] args) {
3.         BinaryTree bt = new BinaryTree();
4.         bt.root = bt.new Node(10);
5.         bt.root.left = bt.new Node(20);
6.         bt.root.left.left = bt.new Node(50);
7.         bt.root.left.right = bt.new Node(40);
8.         bt.root.right = bt.new Node(39);
9.         bt.root.right.right = bt.new Node(2);
10.        bt.root.right.right.right = bt.new Node(5);
11.        System.out.println(bt.countNodesAtCertainLevel(bt.root,3)); //3
12.    }
13. }

```



التمرين السادس :

حساب عرض الشجرة the maximum number of nodes at any level in the tree

- **فكرة الحل :** مشابهة للتمرين الرابع سنقوم بـ BFS و لكن هذه المرة بدلاً من عد العقد لكل مستوى سنقوم فقط باختزان عدد العقد الأعظمي.

```

1. import java.util.LinkedList;
2. import java.util.Queue;
3. class BinaryTree {
4.     Node root;
5.     class Node {
6.         int value;
7.         Node left, right;
8.         Node(int value) {
9.             this.value = value;
10.            this.left = this.right = null;
11.        }
12.    }

13. public int treeWidth(Node root) {
14.     if (root == null) return 0;
15.     Queue<Node> queue = new LinkedList<>();
16.     queue.add(root);
17.     int max = 0;
18.     while (!queue.isEmpty()) {
19.         int cnt = queue.size();
20.         max = Math.max(max, cnt);
21.         for (int i = 0; i < cnt; i++) {
22.             Node node = queue.poll();
23.             if (node.left != null) queue.add(node.left);
24.             if (node.right != null) queue.add(node.right);
25.         }
26.         return max;
27.     }

```

```

28. public class Main {
29.     public static void main(String[] args) {
30.         BinaryTree bt = new BinaryTree();
31.         bt.root = bt.new Node(10);
32.         bt.root.left = bt.new Node(20);
33.         bt.root.left.left = bt.new Node(50);
34.         bt.root.left.right = bt.new Node(40);
35.         bt.root.right = bt.new Node(39);
36.         bt.root.right.right = bt.new Node(2);
37.         bt.root.right.right.right = bt.new Node(5);
38.         System.out.println(bt.treeWidth(bt.root)); //3
39.     }
40. }

```



التمرين السابع: احسب ارتفاع الشجرة (the longest path of nodes) tree height

- **فكرة الحل:** عند كل عقدة سنقوم بجمع 1 (العقدة الحالية) و نضيف القيمة الأكبر بين أطول مسار على يمين هذه العقدة و أطول مسار على يسار هذه العقدة و هذا يتم بشكل عودي شرط التوقف : عند الوصول لعقدة null سنقوم بإرجاع 1- لأننا نريد أن نصل لعدد الوصلات وليس عدد العقد بالمسار فبدل بدء العد من الـ 0 نبدأ من الـ 1.

```

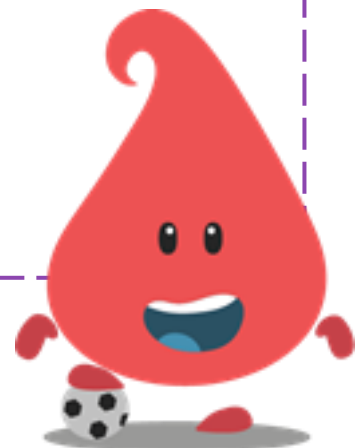
1. import java.util.LinkedList;
2. import java.util.Queue;
3. class BinaryTree {
4.     Node root;
5.     class Node {
6.         int value;
7.         Node left, right;
8.         Node(int value) {
9.             this.value = value;
10.            this.left = this.right = null;
11.        }
12.    }
13.    public int treeHeight(Node root) {
14.        if (root == null) return -1;
15.        // ارتفاع شجرة فارغة هو -1
16.        return 1 + Math.max(treeHeight(root.left),
17.                             treeHeight(root.right));
18.    }
19. }

```

```

18. public class Main {
19.     public static void main(String[] args) {
20.         BinaryTree bt = new BinaryTree();
21.         bt.root = bt.new Node(10);
22.         bt.root.left = bt.new Node(20);
23.         bt.root.left.left = bt.new Node(50);
24.         bt.root.left.right = bt.new Node(40);
25.         bt.root.right = bt.new Node(39);
26.         bt.root.right.right = bt.new Node(2);
27.         bt.root.right.right.right = bt.new Node(5);
28.         System.out.println(bt.treeHeight(bt.root)); //3
29.     }
30. }

```



التمرين الثامن : هل الشجرة المعطاة خطية linear binary tree

متى تكون الشجرة الثنائية خطية ؟

عندما تمتلك كل عقدة ابناً واحداً على الأكثر (تصبح تشبه الـ linked list).

- **فكرة الحل :** عند كل عقدة نتحقق هل لها ابن واحد ، إذا كان نعم نستدعي التابع على أبنائها (حتى وإن كانت عقدة null ليس بمشكلة) بشكل عودي ، سنذكر شروط التوقف ، شروط التوقف :
 - إذا كانت العقدة null نرجع true كوننا وصلنا لنهاية الشجرة فلا يجب أن تؤثر قيمة null على إجابتنا
 - شرط توقف آخر عند الوصول لعقدة ليس لها ابن واحد فقط لدينا حالتان :
 - إما أن يكون لها ابناء و هنا نرجع false لأنها خرقت شرط الخطية.
 - أو يكون ليس لها أبناء فنرجع true كونها آخر عقدة بالمسار و لا تخرق شرط الخطية .

```

1.import java.util.LinkedList;
2.import java.util.Queue;
3.class BinaryTree {
4.  Node root;
5.  class Node {
6.    int value;
7.    Node left, right;
8.    Node(int value) {
9.      this.value = value;
10.     this.left = this.right = null; } }

```

```

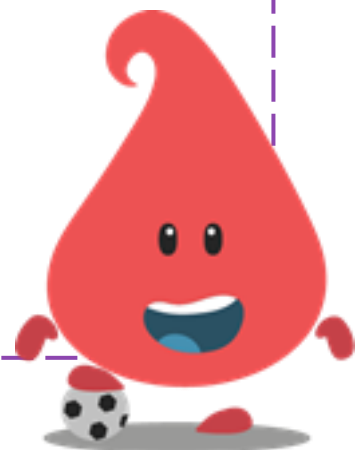
11. public boolean isLinearTree(Node root) {
12.   if (root == null) return true;
13.   if ((root.left==null&& root.right!=null) ||
14.       (root.left !=null &&root.right==null))
15.     return isLinearTree(root.left) &&
16.            isLinearTree(root.right);
17.   return root.left==null&&root.right== null;
18. }

```

```

18.public class Main {
19.  public static void main(String[] args) {
20.    BinaryTree bt = new BinaryTree();
21.    bt.root = bt.new Node(10);
22.    bt.root.left = bt.new Node(20);
23.    bt.root.left.left = bt.new Node(50);
24.    bt.root.left.right = bt.new Node(40);
25.    bt.root.right = bt.new Node(39);
26.    bt.root.right.right = bt.new Node(2);
27.    bt.root.right.right.right = bt.new Node(5);
28.    System.out.println(bt.isLinearTree(bt.root)); //3
29.  }
30. }

```



التمرين التاسع : هل الشجرة المعطاة شجرة مكتملة complete binary tree

متى تكون الشجرة مكتملة ؟

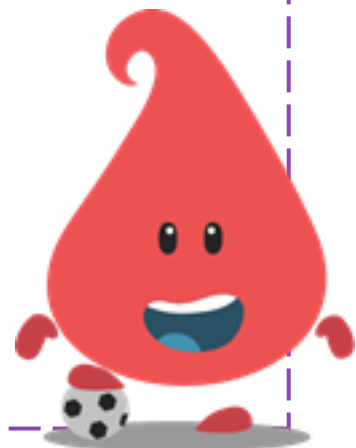
عندما يكون لكل عقدة في الشجرة ابنان إلا في المستوى الأخير و إن حدث نقص فإنه يجب أن يكون من اليمين.

- **فكرة الحل :** نذكر أنه يمكننا ترقيم العقد في الشجرة باستخدام indexing method حتى نقوم بتمثيلها بمصفوفة بحيث عند كل عقدة i فإن الابن الأيسر يمكن ترقيمه بـ $2*i$ و الابن الأيمن بـ $2*i+1$ و سنستفيد من هذا المفهوم نحتاج لعدد العقد الكلي لدينا، تعلمنا حسابه في التمرين الأول ، بحيث من خلال عددها يمكننا التنبؤ بما هو ترقيم آخر عقدة في هذه الشجرة و الذي لا يمكن لأي عقدة تجاوزه كون (لو مثلاً لدينا 7 عقد فأخر index يجب أن تكون فيه عقدة هو 7th index) و هذا يعود إلى أن تزايد الـ indexes يكون level by level من اليسار إلى اليمين فعدد العقد يمكن أن يعطينا تصوراً عن مكان آخر عقدة من الممكن لها أن توجد ضمن هذه الشجرة في حال كانت مكتملة .
- > سنقوم بتتبع الـ index عند كل عقدة في الشجرة نرى إذا كانت الشجرة على يسارها مكتملة و الشجرة على يمينها مكتملة و نقوم بهذا بشكل عودي < ، شرط التوقف :
- إذا وصلنا لعقدة null نعيد true.
- إذا وصلنا لعقدة الـ index الخاص فيها أكبر من عدد عقد الشجرة (يعني أن العقدة التي حصلنا عليها خارج إطار الـ index المتوقع لها) نرجع false .

```

1. class BinaryTree {
2.     Node root;
3.     class Node {
4.         int value;
5.         Node left, right;
6.         Node(int value) {
7.             this.value = value;
8.             this.left = this.right = null; } }
9.     public boolean isCompleteTree(Node root, int index, int last) {
10.         if (root == null) return true;
11.         if (index > last) return false;
12.         return isCompleteTree(root.left, 2 * index, last) &&
13.             isCompleteTree(root.right, 2 * index + 1, last);
14.     public int size(Node root) {
15.         if (root == null) return 0;
16.         return 1 + size(root.left) + size(root.right); } }

```



```

17. public class Main {
18.     public static void main(String[] args) {
19.         BinaryTree bt = new BinaryTree();
20.         bt.root = bt.new Node(10);
21.         bt.root.left = bt.new Node(20);
22.         bt.root.left.left = bt.new Node(50);
23.         bt.root.left.right = bt.new Node(40);
24.         bt.root.right = bt.new Node(39);
25.         bt.root.right.right = bt.new Node(2);
26.         bt.root.right.right.right = bt.new Node(5);
27.         System.out.println(bt.isCompleteTree(bt.root,1, bt.size(bt.root)));//false
28.         bt.root.right.right.right = null;
29.         bt.root.right.right = null;
30.         bt.root.right.left =bt.new Node(5);
31.         System.out.println(bt.isCompleteTree(bt.root,1, bt.size(bt.root)));//true
32. }}

```

التمرين العاشر و الأخير : لدينا مصفوفتان يعبران عن preorder and inorder representation of a tree
مهمتك هي إعادة بناء هذه الشجرة بناء على هذين التمثيلين.

- **فكرة الحل :** لدينا المصفوفة تمثل ال preorder traversal و الذي من خلاله كنا نعالج الجذر أولاً ثم نذهب إلى الطرف الأيسر ثم الطرف الأيمن و المصفوفة الأخرى تمثل ال inorder traversal التي نعالج من خلالها الطرف الأيسر ثم الجذر ثم الطرف الأيمن حتى نبني الشجرة الأصلية بشكلها المطلوب سنعتمد على نقطتين:
 - نحن نعرف أن العقدة الأولى في مصفوفة preorder تمثل الجذر الأول و الذي سنطلق منه كونه الجذر نضيفه كعقدة أولى للشجرة
 - و بعد ذلك سنبحث عن هذا الجذر في مصفوفة inorder و الهدف من ذلك أن نعرف ما هي عناصر الشجرة اليسرى و ما هي عناصر الشجرة اليمنى بحيث يمكننا حصر المجالات فكل العناصر قبل الجذر في مصفوفة inorder تقع بالشجرة اليسرى و كل العناصر بعد الجذر في مصفوفة inorder تقع بالشجرة اليمنى و بالتالي نستدعي التابع بشكل عودي على الشجرة اليمنى و الشجرة اليسرى و نقوم بنفس الموضوع بالنسبة لكل عقدة .
 - بحيث نعتبرها جذر الشجرة الجديدة و كل العناصر على يمينها ضمن المجال الذي حددناه سابقاً تقع على يمينها و كل العناصر على يسارها ضمن المجال الذي حددناه سابقاً تقع على يسارها و في كل استدعاء نضيف العقدة التالية ضمن مصفوفة pre و نظراً لأن ترتيب سير المصفوفة هذه هو :

(جذر – يسار – يمين) سنقوم ببناء الشجرة اليسرى أولاً ثم بناء الشجرة اليمنى ثانياً بطريقة عودية (هكذا نضمن أن إضافة العقدة التالية ضمن كل استدعاء عودي ستكون في محلها).

شروط التوقف :

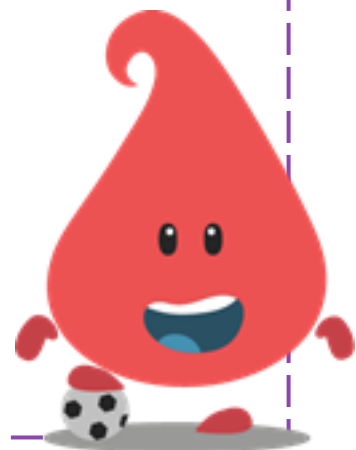
- عندما تصبح بداية المجال بعد نهاية المجال نرجع null (لم يعد هنالك عقد).
- عندما تصبح بداية المجال تساوي نهاية المجال نرجع العقدة الحالية التي نقف عندها دون استدعاءات إضافية .

بعد بناء الشجرة نقوم بإرجاع الـ root في النهاية.

```

1. class BinaryTree {
2.     Node root;
3.     class Node {
4.         int value;
5.         Node left, right;
6.         Node(int value) {
7.             this.value = value;
8.             this.left = this.right = null; } }
9.     int preIndex;
10.    public Node buildTree(int[] inorder, int[] preorder) {
11.        preIndex = 0;
12.        return build(inorder,preorder,0,inorder.length-1);}
13.    private Node build(int[] inorder, int[] preorder, int l, int r) {
14.        if (l > r)
15.            return null;
16.        Node node=new Node(preorder[preIndex]);
17.        preIndex++;
18.        if (l ==r)
19.            return node;
20.        int s = search(inorder,l,r,node.value);
21.        node.left = build(inorder, preorder,l,s -1);
22.        node.right = build(inorder, preorder,s+1, r);
23.        return node; }
24.    private int search(int[] arr, int l, int r, int val) {
25.        for (int i = l; i <= r;i++) {
26.            if (arr[i]==val)
27.                return i;}
28.        return -1;
29.    }}

```




```
29. public class Main {
30.     public static void main(String[] args) {
31.         BinaryTree bt = new BinaryTree();
32.         //      10
33.         //    /  \
34.         //   20   39
35.         //  / \   \
36.         // 50 40  2
37.         //      \
38.         //       5
39.         int[] preorder = {10,20,50,40,39,2,5};
40.         int[] inorder = {50,20,40,10,39,2,5};
41.         bt.root = bt.buildTree(inorder, preorder);
42.         System.out.println(" " + bt.root.value);
43.         System.out.println(" " + bt.root.left.value + " " + bt.root.right.value);
44.         System.out.println(bt.root.left.left.value + " " + bt.root.left.right.value + " " +
45. bt.root.right.right.value);
46.         System.out.println(" " + bt.root.right.right.right.value);
47.     }
48. }
```



THE END